Risolutore Rush Hour con ASP

Delogu Alessandro
Corso di Programmazione Dichiarativa

1 Introduzione e Descrizione del Problema

1.1 Il Gioco

Rush Hour è un rompicapo che simula una situazione di ingorgo stradale in cui il giocatore deve liberare un veicolo intrappolato. Il gioco si presenta come una griglia quadrata di dimensioni 6×6 celle. All'interno di questa griglia sono disposti vari veicoli, che possono essere di due tipi:

- Auto: occupano 2 celle consecutive.
- Camion: occupano 3 celle consecutive.

Ogni veicolo ha un orientamento fisso, determinato alla configurazione iniziale, che può essere:

- Verticale: può muoversi solo in alto o in basso (lungo le colonne).
- Orizzontale: può muoversi solo a destra o a sinistra (lungo le righe).

Tra i vari veicoli, è presenta l'auto rossa, posizionata sempre orizzontalmente sulla terza riga. L'obiettivo del gioco è quello di eseguire una serie di mosse per liberare la strada e permettere all'auto rossa di uscire dalla griglia attraverso un'apertura presente sul bordo destro della griglia stessa, in corrispondenza della terza riga.

Figure 1: Griglia di Gioco con Configurazione Auto

Le regole del gioco impongono dei vincoli sul movimento delle auto, che definiscono la complessità del problema:

- Scorrimento: I veicoli possono solo scorrere avanti o indietro lungo la loro direzione di orientamento.
- Nessuna Rotazione: I veicoli non possono ruotare né spostarsi lateralmente (un'auto verticale non può mai diventare orizzontale).
- Collisioni: I veicoli non possono occupare celle già occupate da altri veicoli.

- Bordi: I veicoli non possono oltrepassare i confini della griglia 6×6 (ad eccezione dell'auto rossa quando raggiunge l'uscita).
- Salti: Non è permesso scavalcare altri veicoli.

La risoluzione di un livello consiste nel trovare una sequenza ordinata di mosse che trasformi lo stato iniziale (la configurazione di partenza) in uno stato finale (dove l'auto rossa è fuori dalla griglia), rispettando tutti i vincoli sopra elencati.

1.2 Obiettivo del progetto

Dal punto di vista formale, Rush Hour può essere visto come un problema di planning in cui bisogna rispettare una serie di regole per trovare la soluzione. Per questo motivo, il problema si presta perfettamente per essere risolto tramite un approccio dichiarativo. L'obiettivo di questo progetto è modellare il rompicapo utilizzando il formalismo dell'Answer Set Programming (ASP). Nello specifico, il lavoro si concentra su:

- Modellazione: Definire il dominio del gioco tramite predicati (per rappresentare auto, posizioni e tempo) e vincoli di integrità (per gestire le collisioni e i limiti della griglia).
- Risoluzione e Ottimizzazione: Sfruttare il solver clingo per calcolare una sequenza di mosse valida che permette all'auto rossa di uscire. Provando ad ottimizzare, l'obiettivo non è trovare una soluzione qualsiasi, ma la soluzione minima.
- Analisi: Testare il modello su diverse istanze per valutare l'efficienza del solver.

2 Codice e Scelte Implementative

In questo capitolo descriveremo la modellazione del problema Rush Hour utilizzando il paradigma dell'Answer Set Programming. Inizialmente è stato realizzato un modello base in cui ogni auto si muove di una singola cella per step temporale, successivamente il modello è stato raffinato per permettere spostamenti multipli in un'unica azione, riducendo il numero di mosse necessario necessario per trovare una soluzione.

Per garantire la modularità, le istanze dei vari livelli sono state separate dalla logica di risoluzione.

2.1 Modellazione della Griglia e Configurazione

La griglia viene definita come una matrice di celle 6×6 con le righe e le colonne numerate 0..5:

```
riga(0..5).
col(0..5).
cella(R, C) :- riga(R), col(C).
```

Ogni livello definisce la configurazione iniziale dei veicoli e il massimo numero di step ammesso. Le auto sono definite tramite il predicato auto/3 (ID, orientamento, lunghezza) e la loro posizione iniziale tramite posizione/4 (ID, riga, colonna, tempo=0). Si noti come per coordinata del veicolo si intende la cella in alto a destra da esso occupata.

Di seguito è riportato un esempio di configurazione con l'auto rossa (sempre presente e indicata con l'ID "x"), un'auto e un camion:

```
#const steps=20.

auto(a, o, 2).
posizione(a, 0, 0, 0).

auto(o, v, 3).
posizione(o, 0, 5, 0).

auto(x, o, 2).
posizione(x, 2, 1, 0).
```

Si noti come per coordinata del veicolo si intende la cella in alto a destra da esso occupata. Per gestire la fisica del gioco, è fondamentale quindi derivare l'insieme delle celle fisicamente occupate dal veicolo. Il predicato occupata(A, R, C, T) viene calcolato dinamicamente a ogni step T in base all'orientamento e alla lunghezza L del veicolo:

```
% Se l'auto è orizzontale, occupa le celle (R, C+I) per I da 0 a L-1
occupata(A, R, C+I, T) :- auto(A, o, L), posizione(A, R, C, T), I = 0..L-1.

% Se l'auto è verticale, occupa le celle (R+I, C) per I da 0 a L-1
occupata(A, R+I, C, T) :- auto(A, v, L), posizione(A, R, C, T), I = 0..L-1.
```

2.2 Definizione delle mosse

Prima di modellare il concetto di mossa, bisogna introdurre la gestione del tempo. In questo programma il tempo è discreto nell'intervallo 0..steps, dove il numero di step è definito all'interno di ogni istanza di livello. L'introduzione del tempo T ci permette di far variare lo stato della griglia per ogni valore di T.

Per la generazione delle mosse viene utilizzata una Choice Rule con cardinalità 1 per imporre che, ad ogni istante T, venga eseguita esattamente una mossa di una singola cella. Il predicato mossa(A,D,T) definisce in che direzione D deve andare il veicolo A al tempo T:

```
dir(A, (u;d)) :- auto(A, v, L).
dir(A, (l;r)) :- auto(A, o, L).

1 { mossa(A, D, T) : dir(A, D) } 1 :- time(T), T < steps, not fine(T).
```

Verrà generata una mossa per ogni istante di tempo T fino a che non verrà raggiunto il goal fine(T).

Ogni mossa generata al tempo T andrà ad influenzare la posizione delle auto al tempo $T + 1$. Se un'auto si muove, la sua coordinata viene aggiornata mentre se un'auto non viene mossa, mantiene la sua posizione precedente:

```
posizione(A, R+1, C, T+1) :- mossa(A, d, T), posizione(A, R, C, T).
posizione(A, R-1, C, T+1) :- mossa(A, u, T), posizione(A, R, C, T).
posizione(A, R, C+1, T+1) :- mossa(A, r, T), posizione(A, R, C, T).
posizione(A, R, C-1, T+1) :- mossa(A, l, T), posizione(A, R, C, T).
posizione(A, R, C,   T+1) :- not mossa(A, _, T), posizione(A, R, C, T),
                             time(T), T < steps.
```

Per assicurare la validità delle mosse, vengono introdotti dei vincoli di integrità. Nella versione base, viene controllato semplicemente che la cella di destinazione sia valida e libera, in quanto una mossa è di una sola cella. Questi vincoli permettono di gestire:

- Limiti dalla Griglia: È vietato muoversi se la nuova posizione cade fuori dai bordi.
- Collisioni: È vietato che due auto diverse occupino la stessa cella nello stesso istante.

I vincoli di integrità sono così definiti:

```
:- mossa(A, d, T), posizione(A, R, C, T), auto(A, v, L), not cella(R+L, C).
:- mossa(A, u, T), posizione(A, R, C, T), auto(A, v, L), not cella(R-1, C).
:- mossa(A, r, T), posizione(A, R, C, T), auto(A, o, L), not cella(R, C+L).
:- mossa(A, l, T), posizione(A, R, C, T), auto(A, o, L), not cella(R, C-1).

:- occupata(A, R, C, T), occupata(B, R, C, T), A != B.
```

2.3 Definizione del Goal e Ottimizzazione

Il goal è raggiunto quando l'auto rossa x raggiunge la coordinata di uscita (riga 2, colonna 4). Una volta raggiunto il goal, il predicato fine(T) blocca la generazione di ulteriori mosse:

```
fine(T) :- posizione(x, 2, 4, T).
:- not fine(_).
```

Per far si che venga trovata la sequenza minore di mosse per raggiungere il goal, viene utilizzata la direttiva #minimize in funzione di T:

```
#minimize { 1, T : mossa(_,_,T) }.
```

2.4 Modellazione Mosse di N celle

Ora che abbiamo trovato una soluzione ai nostri livelli, seppur ottimizzata non risulta essere ancora la sequenza minima. Questo è dovuto al fatto che ogni mossa può spostare un veicolo di una singola cella. Modellando le mosse in modo tale che possano far muovere i veigoli di più celle alla volta (tutte libere), il numero di mosse finali verrebbe ulteriormente ridotto.

Per fare ciò è stato necessario andare ad agire sulla modellazione delle mosse, introducendo un nuovo parametro che indica il numero di celle coinvolte nella mossa.

```
dist(1..4).

1 { mossa(A, D, N, T) : dir(A, D), dist(N) } 1 :- time(T), T < steps, not fine(T).
```

È stato inoltre necessario intervenire anche sui vincoli di integrità delle mosse per far si che vengano considerate valide solo le tutte le celle coinvolte in esse sono libere:

```
:- mossa(A, d, N, T), posizione(A, R, C, T), auto(A, v, L),
    K = 1..N, X = R + L-1 + K,
    1 { not cella(X, C); occupata(B, X, C, T) : A != B }.
:- mossa(A, u, N, T), posizione(A, R, C, T), auto(A, v, L),
    K = 1..N, X = R - K,
    1 { not cella(X, C); occupata(B, X, C, T) : A != B }.
```

```
:- mossa(A, r, N, T), posizione(A, R, C, T), auto(A, o, L),
    K = 1..N, X = C + L-1 + K,
    1 { not cella(R, X); occupata(B, R, X, T) : A != B }.
:- mossa(A, l, N, T), posizione(A, R, C, T), auto(A, o, L),
    K = 1..N, X = C - K,
    1 { not cella(R, X); occupata(B, R, X, T) : A != B }.
```

3 Analisi dei risultati

In questo capitolo verranno presentati i dati raccolti durante la fase di testing. L'obiettivo è valutare l'efficienza della modellazione ASP proposta e l'impatto delle diverse strategie di solving offerte da clingo su istanze di complessità crescente.

3.1 Dataset e Configurazioni

Per questo benchmark sono state selezionate 15 istanze tratte dal gioco originale Rush Hour, suddivise in tre categorie di difficoltà:

1. **Beginner (1-5)**: Richiedono tra 8 e 15 mosse. Servono a verificare la correttezza di base.
2. **Advanced (21-25)**: Richiedono tra 16 e 30 mosse. Rappresentano la complessità media.
3. **Hard (35-40)**: Richiedono oltre 30 mosse (fino a 50+). Queste istanze mettono sotto stress lo spazio di ricerca del solver.

3.2 Metriche e Modalità

Le metriche raccolte per ogni esecuzione sono:

1. **Status**: Esito della ricerca (SAT se la soluzione è trovata, UNSAT se non esiste entro l'orizzonte fissato, TIMEOUT se il tempo scade).
2. **Solving Time (s)**: Indica il tempo impiegato dal solver per l'esplorazione dello spazio di ricerca. Questa è la metrica principale per valutare l'efficienza delle euristiche.
3. **Grounding Time (s)**: Il tempo impiegato per istanziare le variabili.
4. **Steps**: Numero di mosse che compone la soluzione minima.

I test sono stati condotti in modalità Single-shot, ovvero lanciando il solver da zero per ogni istanza con un orizzonte temporale sufficiente, nelle seguenti configurazioni:

- **Single Thread**: Esecuzione standard su un singolo core.
- **Multi Thread**: Esecuzione parallela su 4 core (–parallel-mode=4).
- **Solver Config**: Confronto tra i preset euristici di clingo (tweety, trendy, frumpy, crafty, jumpy, handy).

3.3 Single Thread vs Multi Thread

Questo grafico evidenzia il vantaggio del Multi-threading. Sulle istanze difficili la linea del Multi Core rimanere più bassa rispetto al Single Core, dimostrando l'efficacia dell'approccio parallelizzato.

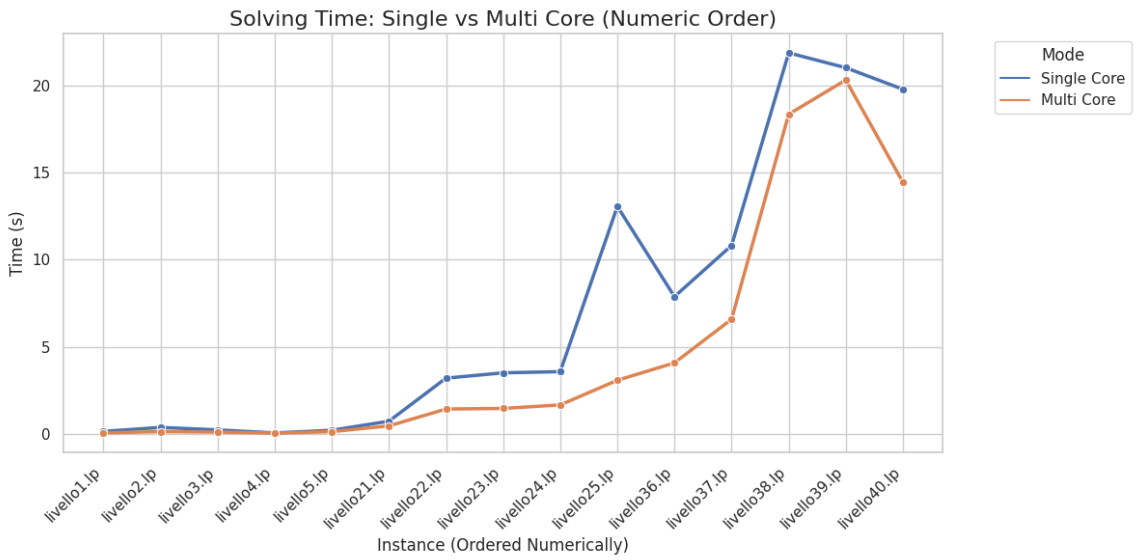

Figure 2: Benchmark Solving Time Single-Core vs Multi-Core

3.4 Configurazioni del Solver a Confronto

Questo grafico mostra il confronto tra le varie configurazioni che il solver clingo mette a disposizione. Si può vedere come le configurazioni frumpy e crafty non sono adatte a qeusto tipo di problemi mentre le altre quattro ottengono risultati simili.

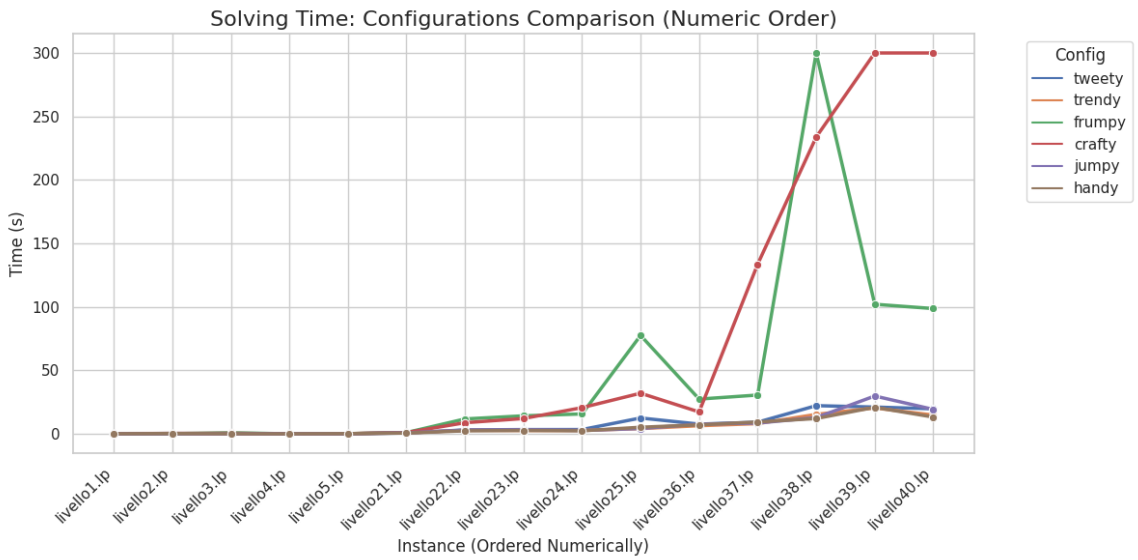

Figure 3: Benchmark Configurazioni Solver a Confronto

3.5 Limiti dell'Approccio Single Shot

L'approccio utilizzato in questo progetto è di tipo Single-Shot. Per cercare una soluzione, il problema viene istanziato interamente con un orizzonte temporale fisso. Questo può essere limitante in quanto bisogna sapere prima entro quante mosse il problema può essere risolto.

Un approccio Multi-Shot incrementale rappresenterebbe l'evoluzione naturale per questo tipo di problema. Il solving multi-shot permette di gestire programmi logici che evolvono nel tempo. Un approccio incrementale invece di rilanciare il grounder da zero, si mantiene in memoria la parte statica del problema (la griglia, le regole di movimento) e si aggiunge iterativamente solo il nuovo step temporale. Questo permette di trovare la soluzione minima incrementando invece che trovarla minimizzando una soluzione ammissibile generale trovata.

4 Conclusioni

Il progetto ha permesso di analizzare l'efficacia dell'Answer Set Programming (ASP) applicata a un problema di pianificazione combinatoria con vincoli geometrici come il gioco "Rush Hour".

Attraverso la modellazione del dominio, siamo passati da una rappresentazione iniziale a passo singolo a una più efficiente che permette mosse multiple, riducendo significativamente l'orizzonte temporale necessario per trovare la soluzione.

L'analisi dei risultati ci ha permesso di osservare come tramite un'approccio multi shot e tramite una corretta configurazione del solver, sia possibile ottimizzare il Solving Time.